

# Intro Programmeren

## INFO

### Lesinhoud

Deze les helpt je om te denken als een programmeur: je leert hoe je een probleem analyseert, het opsplitst in kleinere stappen en een oplossing uitwerkt in duidelijke instructies. We gebruiken pseudocode om die instructies op een eenvoudige en begrijpelijke manier te noteren, zonder dat je al echte code hoeft te schrijven.

### Doelstellingen

- een logisch probleem opdelen in kleine onderdelen om tot een oplossing te komen.

## ▼ Inhoud

- [Introductie: de computer als gehoorzame assistent](#)
- [Wat is een algoritme? De recept-analogie](#)
- [Van probleem naar oplossing in 3 stappen](#)
- [Praktijkvoorbeeld: koffie zetten](#)
- [Een blik vooruit: kernconcepten zonder code](#)
  - [Variabelen: een doos met een etiket](#)
  - [Conditie: als dit => dan dat](#)
  - [Herhalingen: blijf herhalen tot het klaar is](#)
  - [Lijsten: een rek met genummerde vakjes](#)
  - [Functies: een herbruikbaar recept](#)

---

## Introductie: de computer als gehoorzame assistent

Welkom bij de start van je reis in de wereld van het programmeren!

Voordat we ons verdiepen in JavaScript, is er een nog belangrijkere vaardigheid die je moet leren: denken als een programmeur en leren problemen te analyseren.

Stel je een computer voor als een extreem krachtige, maar tegelijkertijd niet zelfstandige assistent. Hij voert duizenden berekeningen per seconde uit, maar neemt geen initiatief en heeft geen gezond verstand. Hij doet **enkel en alleen** wat jij hem opdraagt, in de **exacte volgorde** die jij bepaalt. Als je een stap vergeet of je een vage instructie geeft, dan zal de assistent in de war raken of gewoon niets doen.

**Programmeren is dus niet alleen het schrijven van code, maar vooral het leren geven van duidelijke, logische en ondubbelzinnige instructies.**

In dit hoofdstuk leer je hoe je zo'n stappenplan opstelt, zodat je deze assistent slim en efficiënt kunt inzetten.

Verder gaan we ook gebruik maken van **Blockly** om de basisprincipes op een eenvoudige en visuele manier te verkennen. Na het oplossen van een oefening krijg je ook de JavaScript code te zien. Op die manier kan je je een idee vormen over hoe dit er later in de code gaat uitzien.

**Scratch** is een andere visuele programmeeromgeving waarmee je de basisconcepten van programmeren kan leren. Hierover vind je een aparte introductie en vervolgens komen er ook enkele oefeningen aan bod.

Hier komt alvast een eerste opwarmer van **Blockly**:

▼ **Oefening - Puzzel**

Ga naar <https://blockly.games/?lang=nl> en los de **puzzel** opdracht op.

*Voorziene tijd: 10 minuten*

---

## Wat is een algoritme? De recept-analogie

Een **algoritme** is een stappenplan waarmee je een computer vertelt hoe hij een taak moet uitvoeren. Dit klinkt misschien moeilijk, maar eigenlijk gebruik je dagelijks algoritmes zonder het te beseffen. Denk maar aan een recept om een cake te bakken - dit is een perfect voorbeeld van een algoritme.

Laten we het vergelijken:

- **De ingrediënten** (bloem, suiker, eieren): Dit is de **input** van je algoritme. Zonder deze beginwaarden kan je niet starten.
- **De instructies** ("Verwarm de oven voor op 180°C", "Meng de droge ingrediënten"): Dit zijn de opeenvolgende stappen die je moet volgen.
- **De volgorde** is hierbij cruciaal. Je kan de cake niet uit de oven halen voordat je het deeg hebt gemengd en in de oven hebt geplaatst.
- **De afgewerkte cake**: Dit is de **output**, het resultaat van je stappenplan.

Een goed algoritme is dus zoals een goed recept: duidelijk, volledig en in de juiste volgorde. Als je een stap overslaat of de volgorde verandert, krijg je geen lekkere cake en in de programmeerwereld krijg je dan een foutmelding of een programma dat niet werkt.

▼ **Engelstalige video over algoritme - OPTIONEEL**

What exactly is an algorithm? Algorithms explained ...



---

## Van probleem naar oplossing in 3 stappen

Elk probleem lijkt in het begin misschien complex, maar om tot een goed algoritme te komen, kan je een eenvoudig proces van drie stappen volgen. Deze methode helpt je om elk probleem, hoe groot ook, stap voor stap aan te pakken en om te zetten naar een duidelijk oplossing.

### Stap 1: Begrijp het probleem en herformuleer het eenvoudig

Wat is het *echte* doel? Wat probeer je te bereiken?

Probeer het probleem in je eigen woorden uit te leggen. Als je het niet eenvoudig kan uitleggen, begrijp je het waarschijnlijk nog niet volledig.

### Stap 2: Identificeer input, verwerking en output

Elk programma werkt volgens hetzelfde principe: het neemt wat gegevens (**input**), doet er iets mee (**verwerking**), en geeft een resultaat terug (**output**).

- **Input:** Welke gegevens of materialen heb je nodig om te starten?
- **Verwerking:** Wat moet er gebeuren met die gegevens? Welke acties of berekeningen zijn nodig?
- **Output:** Wat wil je uiteindelijk tonen of bekomen?

### Stap 3: Werk het stappenplan uit in pseudocode

Voordat je echte code schrijft, noteer je je oplossing in pseudocode: een soort mensentaal die de logica van je programma beschrijft zonder technische details. Zo krijg je een helder stappenplan (algoritme) dat je later makkelijk kunt omzetten in echte code. Het is een blauwdruk van je programma, zonder dat je je zorgen moet maken over de precieze schrijfwijze (zoals puntkomma's of haakjes).

---

## Praktijkvoorbeeld: koffie zetten

Laten we de 3-stappenmethode toepassen op een alledaags probleem: "Ik wil een tas koffie zetten met een standaard koffiezetapparaat."

**Stap 1: Begrijp en herformuleer het probleem** Het doel is om van losse ingrediënten (water, koffie) naar een drinkbare tas hete koffie te gaan met behulp van een machine.

### Stap 2: Bepaal input, verwerking en output

- **Input:** Koffiepoeder, een papieren filter, water, een koffiekan, een tas.
- **Verwerking:** Vullen, aanzetten, opwarmen en doorlopen van het water door de koffie.
- **Output:** Een tas hete koffie.

### Stap 3: Schrijf het stappenplan (pseudocode)

1. NEEM een papieren filter uit de doos.
2. PLAATS de filter op de juiste manier in de

filterhouder van het apparaat.

3. VOEG 2 schepjes koffiepoeder toe aan de filter.

4. VUL de koffiekan met water tot aan de markering voor 1 tas.

5. GIET het water uit de kan in het waterreservoir van het apparaat.

6. PLAATS de lege kan terug op de warmhoudplaat.

7. DRUK op de 'start'-knop.

8. WACHT totdat de kan volledig gevuld is met koffie.

9. NEEM een tas uit de kast en plaats deze op zijn onderkant op de tafel.

10. SCHENK de koffie uit de kan in de tas tot hij bijna vol is.

Dit stappenplan is helder en volledig. Een assistent zonder kennis van koffie zetten kan dit nu perfect uitvoeren. Dit is precies de denkwijze die je nodig hebt om te leren programmeren.

▼ Engelstalige video over instructies - OPTIONEEL

Exact Instructions Challenge PB&J Classroom Frien...



▼ Oefening - Doolhof

Ga naar <https://blockly.games/?lang=nl> en los de **Doolhof** opdracht op. Hier leer je met logische stappen en herhalingen werken.

*Voorziene tijd: 20 - 30 minuten*

---

## Een blik vooruit: kernconcepten zonder code

De logische stappen die je in pseudocode beschrijft, komen overeen met de basisbouwstenen van elke programmeertaal. Laten we er een paar bekijken aan de hand van eenvoudige analogieën.

---

### Variabelen: een doos met een etiket

In ons koffie-recept hebben we 'koffiepoeder' en 'water'. In een programma moeten we dit soort informatie tijdelijk kunnen bewaren. Een **variabele** is als een doos waar je iets in stopt en waar je een etiket op plakt. Zo kan je later makkelijk naar de inhoud verwijzen.

We kunnen een doos genaamd `aantalSchepjesKoffie` hebben met de waarde `2` erin, of een doos `isKoffieKlaar` met de waarde `nee`.

- `aantalSchepjesKoffie` = 2
- `isKoffieKlaar` = nee

---

### Conditie: als dit => dan dat

Soms moet een programma een keuze maken. Dit noemen we een **conditie**.

- **ALS** het regent, **DAN** neem ik een paraplu mee.
- **ANDERS** (als het niet regent), **DAN** neem ik een zonnebril mee.

Dit 'als-dan-anders'-denken is een cruciaal onderdeel van programmeren. In Javascript zal je dit vertalen naar een `if` - `else` - structuur.

#### ▼ Oefening - Vogel

Ga naar <https://blockly.games/?lang=nl> en los de **Vogel** opdracht op. Hier gebruik je condities om een vogel naar zijn nest te leiden.

*Voorziene tijd: 15 - 20 minuten*

---

## Herhalingen: blijf herhalen tot het klaar is

Soms moet je een actie meerdere keren herhalen. **ZOLANG** er vuile borden op het aanrecht staan, **HERHAAL**: pak een bord, was het af, en zet het in het droogrek. Je stopt pas als de voorwaarde ("er zijn vuile borden") niet meer waar is.

Dit concept van herhaling zal je later tegenkomen als **loops** (bv. `for`, `forEach` of `while` -loops).

#### ▼ Oefening - Schildpad

Ga naar <https://blockly.games/?lang=nl> en los de **Schildpad** opdracht op. Herhalingen werden bij de doolhof opdracht ook al behandeld. We maken het hier iets moeilijker. Je gaat namelijk figuren tekenen met herhalingen en wiskundige logica.

*Voorziene tijd: 20 - 30 minuten*

---

## Lijsten: een rek met genummerde vakjes

In een programma wil je vaak meerdere stukjes informatie van hetzelfde type bijhouden — bijvoorbeeld de namen van studenten, de prijzen van



producten of de kleuren van een ontwerp. In plaats van voor elk item een aparte doos (variabele) te maken, gebruik je een lijst.

Een lijst is als een rek met genummerde vakjes. Elk vakje bevat één waarde, en je kunt er makkelijk iets aan toevoegen, eruit halen of opzoeken.

*Bijvoorbeeld:* een besteklade

Stel je een besteklade voor met vakjes:

- Vakje 0: vork
- Vakje 1: mes
- Vakje 2: lepel

Je kunt dan zeggen: "Geef me het item in vakje 1" - dat is het mes.

Lijsten zijn overal in programmeren. Ze maken je code overzichtelijk en flexibel. Of je nu een reeks afbeeldingen toont, een wachtrij bijhoudt, of data verwerkt. Later in de cursus zal je dit leren kennen als een `array`.

---

## Funcities: een herbruikbaar recept

Stel dat je elke ochtend koffie zet. In plaats van telkens het hele stappenplan opnieuw te schrijven, kun je het bundelen in één blok: een functie.

### ▼ Oefening - Film

Ga naar <https://blockly.games/?lang=nl> en los de **Film** opdracht op. Hier gooien we alles samen en ga je animaties programmeren met variabelen en functies.

*Voorziene tijd: 20 - 30 minuten*

---

## Conclusie

In dit hoofdstuk heb je nog geen code geschreven, maar je hebt wel de belangrijkste vaardigheid van een programmeur geoefend: het **systematisch analyseren en oplossen van problemen**.

Onthoud de **3-stappenmethode** :

1. Begrijp het probleem
2. Bepaal de input, verwerking en output
3. Schrijf een helder stappenplan.

Deze manier van denken is cruciaal om later de JavaScript schrijfwijze of *syntax* vlot onder de knie te krijgen. De code is slechts een hulpmiddel - het logische stappenplan is het echte meesterwerk.

Dit is in essentie de basis van programmeren: **een probleem analyseren - het opdelen in logische, kleinere stappen - en die efficiënt omzetten tot een werkende oplossing**.

---

Previous page  
**Overzicht**

Next page  
**Oefeningen**